ROS2 Spring Workshop

Lab 1: Environment Setup & Linux Survival

Objective

Robotics software requires a specific, isolated environment to run correctly. Instead of wiping your laptop to install Linux, we will use **Docker**. This allows us to run a "virtual robot computer" inside your Windows or Mac laptop.
By the end of this lab, you will have a fully functioning **ROS 2 Humble** environment and will be able to control a simulated turtle.

Pre-Lab Concept Primer: The Virtual Laboratory

Before we install anything, we need to understand **virtualization** and the **command line**.

1. What is a Container?

Robotics software (ROS 2) loves Linux (Ubuntu). Most of you use Windows or macOS. To solve this, we use **Docker containers**.

- **Virtual Machine (VM) = a house.** It has its own foundation, plumbing, and walls. It is heavy and slow.
- **Container = an apartment.** It uses the **Docker Engine** to share a single lightweight foundation, rather than building a whole new house for every robot. This layered relationship is shown in Figure 1.

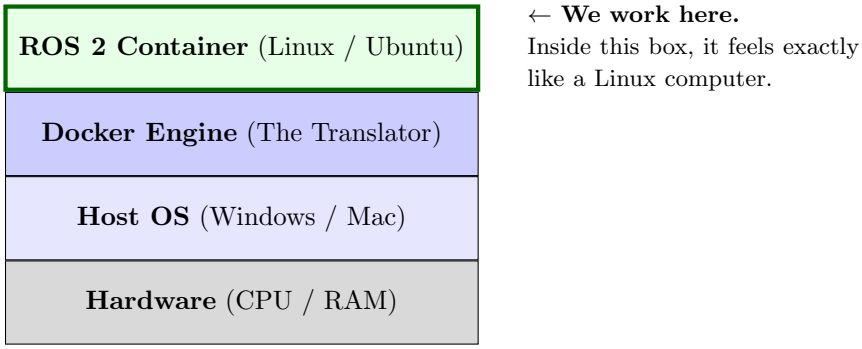

Figure 1: The layers: your laptop runs Docker, and Docker runs ROS 2.

2. Why the Terminal (CLI)?

In this course, you will type commands rather than clicking buttons.

- **Precision:** Clicking "drag" with a mouse is imprecise. Typing `move_x: 5.0` is exact.
- **Scriptability:** You cannot script a mouse click easily. You can script text commands.
- **Remote access:** Real tractors don't have screens in the way we are using them. We talk to them over a text-based network connection (SSH) — something you will do directly in Lab 9.

Part 1: The Hardware Check (Critical)

Before installing software, you must ensure your computer's CPU allows virtualization. We are checking the foundation from Figure 1 to make sure it can support the other layers.

Windows Users:

1. Open Task Manager (`Ctrl + Shift + Esc`).
2. Click the **Performance** tab → **CPU**.
3. Look at the bottom right for **Virtualization**.
4. If it says **Enabled**, proceed to Part 2.
5. If it says **Disabled**, you must restart your computer, enter BIOS (F2/F12/Del), and enable `Intel VT-x` or `AMD-V`.

Mac Users:

- If you have an M1/M2/M3 chip, you are supported by default. Proceed to Part 2.

Linux Users:

- Open your terminal and run: `lscpu | grep Virtualization`
- If you see `VT-x` or `AMD-V`, you are ready. Native Linux users usually have this enabled by default.

Part 2: Docker, VS Code Installation

Install the following tools in this exact order. Here we are installing the Docker Engine (blue layer) and allowing the ROS 2 container to run in VS Code using Dev Containers (green layer) from Figure 1.

Step A: Docker

Docker provides the isolated environment needed for ROS 2.

- **Windows/Mac:** Download and install **Docker Desktop** from the official website.
  - *Windows Note:* Ensure "Use WSL 2 instead of Hyper-V" is **CHECKED**.
  - Windows users will run into issues later if WSL has not been installed before, along with an operating system. If this is the case go to **Appendix B**.
- **Linux (Ubuntu/Debian):** Install the engine and set permissions via terminal:

  Note: If you already have Docker installed, run `docker run hello-world`. If it works, you can skip to Step B. If you get a permission error, run the `usermod` command below.

```
sudo apt update && sudo apt install docker.io -y
sudo usermod -aG docker $USER

```

- **ACTION: Restart or Log Out/In** your computer immediately after installation to apply these system changes.

For the Curious: Why was Windows harder?

Mac computers are built on UNIX, the same "language" as Linux. They can talk to the robot natively. Windows computers speak a different language (NT), so we had to install a translator (Ubuntu via WSL) to make the connection work. You just built a bridge between two operating systems!

Step B: VS Code & Extensions

This is the interface we will use to navigate the hardware later in the course.

- **Windows/Mac:** Download and run the installer from `code.visualstudio.com`.
- **Linux:** Use your preferred method to install **VS Code**.

- **The recommended way (Snap):** This is the most reliable method for this course as it handles security keys and system permissions automatically:

```
# If 'snap' command is not found, run: sudo apt update && sudo apt install snapd -y
sudo snap install --classic code

```

- *Note: Avoid the "Flatpak" version of VS Code, as its security sandbox often prevents it from connecting to Docker.*

- **Extensions:** Open VS Code, click the Extensions icon (Ctrl+Shift+X), and install the **Dev Containers** extension by Microsoft as seen in Figure 2.

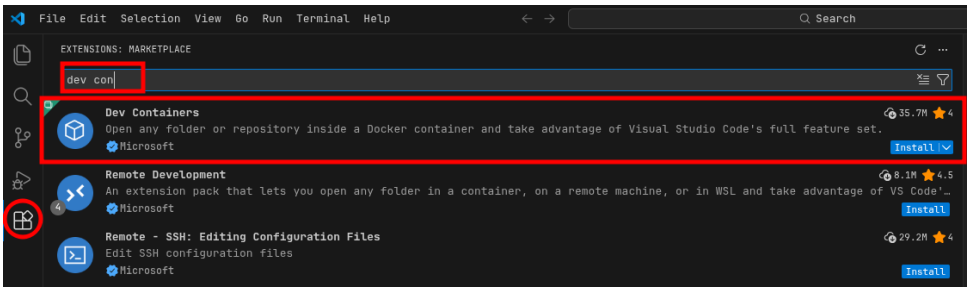

Figure 2: Visual guide: look for the Dev Containers extension by Microsoft in the marketplace.

Part 3: Cloning the Course Material

Congratulations! You have successfully built your virtual laboratory. Whether you are on Windows, Mac, or Linux, you are now operating inside the exact same environment with identical Python versions and ROS 2 dependencies. This "zero to hero" setup ensures that we can focus on agriculture robotics without fighting installation errors. Now, let's grab the actual course materials and get to work.

We will use Git to download the course "skeleton code."

1. Create a folder on your computer named `AgTech_Course`.
   Pro-tip: put this in your Home directory, not in a cloud-sync folder like OneDrive to avoid hair pulling and performance lag!
2. Open VS Code → **File** → **Open Folder** → select `AgTech_Course`.
3. Open the Terminal in VS Code (`Ctrl + ` ` or View → Terminal).
4. Run the following command:

```
# Verify git is installed
git --version

# Clone the course environment
git clone https://github.com/AgTech-UW/agtech_ros2.git
```

Listing 1: Clone the Repository

Part 4: The "Magic" Launch

This is where we build the "virtual robot computer" defined in your repository.

1. In VS Code, go to **File** → **Open Folder**.
2. **CRITICAL:** Navigate *inside* your `AgTech_Course` folder and select the newly created `agtech_ros2` folder.
3. You should see a popup in the bottom right: *"Folder contains a Dev Container configuration file. Reopen to develop in a container."*
4. **Click "Reopen in Container".**

5. *Troubleshooting:* If the popup doesn't appear, click the **green button** (the >< icon) in the bottom-left corner of VS Code and select **Reopen in Container**.

Wait 5–10 minutes. Your laptop is now downloading the ROS 2 "brain" (approx. 3 GB) and running the course bootstrap script. When the process is complete, your window should look similar to Figure 3.

What's Happening During That 5–10 Minute Wait?

On first open, the container does three things in sequence:

1. **Pulls the ROS 2 image** (`osrf/ros:humble-desktop-full`) — this is the 3 GB download.
2. **Builds your dev container** — sets up the workspace and user environment.
3. **Runs** `bootstrap.sh` — installs extra ROS packages you'll need later (camera drivers, AprilTag detection, calibration tools), configures Cyclone DDS for reliable networking, and plants an easter egg for Part 6.

You don't need to do anything during this wait. If you see a lot of `apt-get` output flashing by, that's bootstrap installing Lab 5 and Lab 6 dependencies up front so you don't have to later.

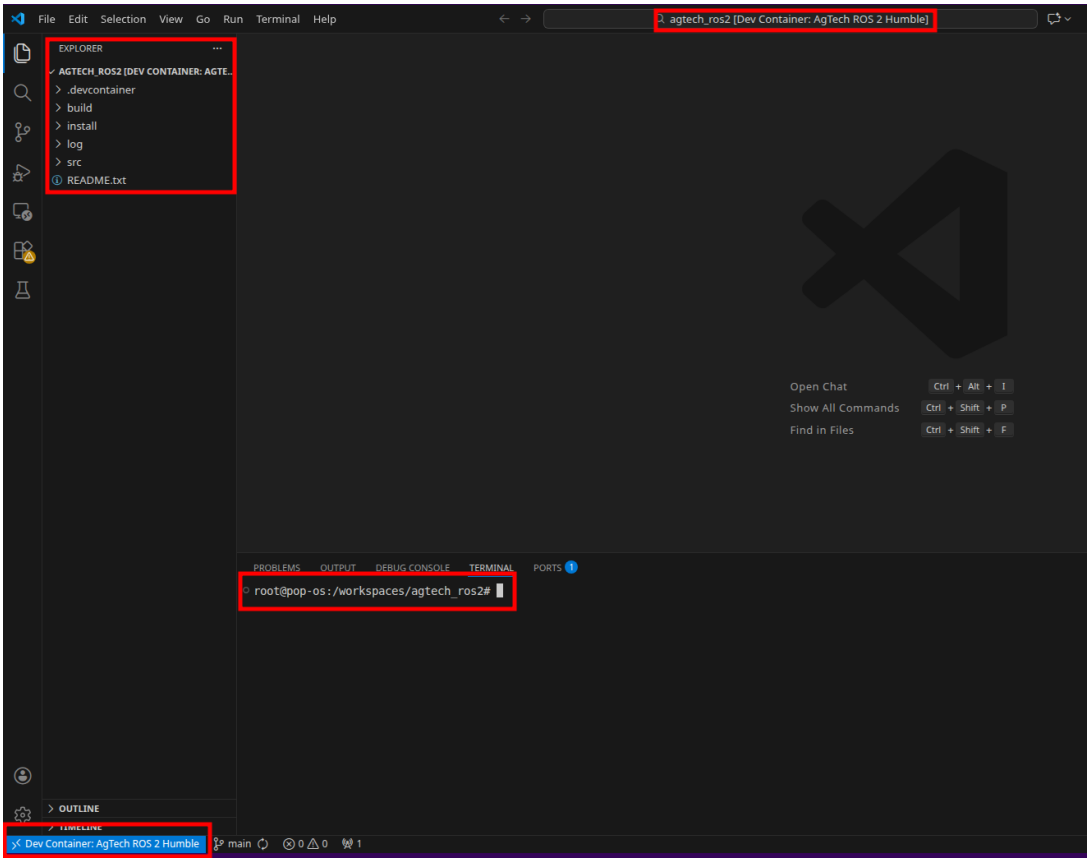

Figure 3: Success: the bottom-left corner should turn blue/green and show "Dev Container," and your terminal prompt will change to `root@hostname`.

Notice that your terminal prompt has changed to:

```
root@hostname:/workspaces/agtech_ros2#
```

Part 5: Verification (Turtlesim)

Now we verify that ROS 2 is active and communicating between nodes.

Test 1: The Environment

Type `ros2` in the terminal. You should see a help menu. If you see "command not found," raise your hand.

Test 2: The Simulation

We will run the standard ROS 2 turtle simulator to test graphical output and communication.

1. In the VS Code terminal, run the simulation node:

```
ros2 run turtlesim turtlesim_node
```

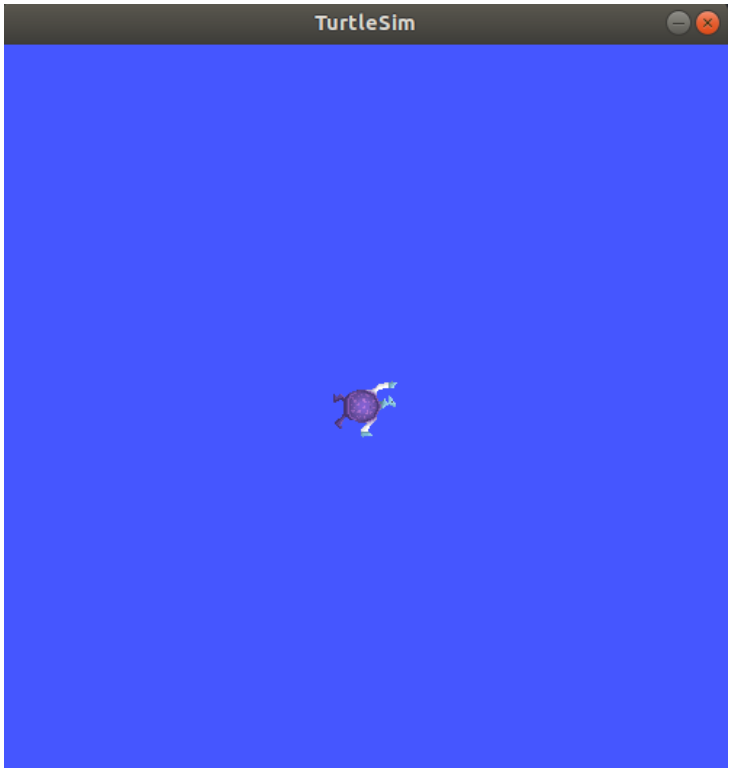

Figure 4: The Turtlesim window: if you see a blue screen with a turtle, your ROS 2 environment is working!

> **Syntax Spotlight: Command Anatomy**
>
> `ros2 run <package> <executable>`
>
> - **ros2:** The main tool.
> - **run:** The verb (action).
> - **turtlesim:** The package (the folder where the code lives).
> - **turtlesim_node:** The executable (the specific script to run).

2. A blue window with a turtle should appear (Figure 4).
3. Open a **second terminal** (split terminal icon or `Ctrl+Shift+5`).
4. Run the keyboard controller:

```
ros2 run turtlesim turtle_teleop_key
```

5. **Driving:** Click your mouse inside the *teleop terminal* and use the arrow keys. The turtle will only move if that specific terminal tab is active!

Part 5.1: Linux Display Troubleshooting

If you are on Linux and the `turtlesim` window does not appear despite the terminal saying "Spawning turtle," follow these steps:

1. **The permission:** Open your host terminal and run `xhost +local:root`.
2. **The socket:** Inside the VS Code terminal, run `export DISPLAY=:1`. (Note: if `:1` fails, try `:0`. Display numbers vary depending on how your desktop session was started — there's no "correct" one; it's an empirical test.)
3. **The re-run:** Run the `ros2 run turtlesim turtlesim_node` command again.

If you are required to do this, please review the Technical Appendix: Security and OS Architecture below.

Part 6: Activity: The Linux Scavenger Hunt

Somewhere inside this container, a secret text file has been hidden. Your mission is to use Linux terminal commands to find it and retrieve the code inside.

The clues:

- The file is somewhere inside your home directory (`/root`).
- There is a folder named `mystery_folder`.
- Inside that, a sub-folder named `level_1`.
- The file itself is **hidden** (in Linux, hidden files start with a dot).

Required commands:

- `ls` (list files) vs `ls -la` (list ALL files, including hidden ones).
- `cd` (change directory).
- `cat` (read file content).

Hint: Start with `cd ~` to go to your home directory, then explore from there. The code you find becomes your **Lab 1 secret code** — you'll include it in every lab deliverable this semester.

Deliverables

Take a screenshot of your full desktop showing:

1. The blue Turtlesim window with the turtle.
2. The terminal showing the `ros2 run` commands.
3. The content of the file from the Linux Scavenger Hunt (your secret code).

What's Next: Lab 2

You've built a virtual Linux laboratory and verified that ROS 2 can drive a simulated robot. In Lab 2 you'll write your first **autonomous node** — a Python program that drives the turtle in a square without any human input. The teleop key-press was a warm-up. Starting next lab, *your code* is the driver.

Appendix A: Troubleshooting Survival Guide

1. "Docker Engine Stopped"

Fix: Open Docker Desktop. Wait for the whale icon to turn **green**. Try again.

2. "Virtualization Disabled"

Fix: Enable "Intel VT-x" or "AMD-V" in your computer's BIOS (F2/Del on boot).

3. Mac Users: No Turtle Window

Fix: Install **XQuartz** from xquartz.org. Log out and back in.

4. Windows Users: "Could not connect to display :1"

Fix: Run `wsl --update` in PowerShell (as an admin) and restart Docker.

5. Git Error 403 (Permission Denied)

Fix: Sign out of GitHub in VS Code (profile icon) and sign back in with the correct account.

6. Linux: Qt Error / "xcb" / No Display

If the window won't open:

- **Step 1:** Run `xhost +local:root` in your host terminal.
- **Step 2:** Run `export DISPLAY=:1` in the VS Code terminal (try `:0` if `:1` fails).
- **Step 3:** If that fails, rebuild container (see below).

7. The Bootstrap Failed Partway

If the 5–10 minute container build succeeds but some packages are missing (e.g., `ros2 pkg list | grep usb_cam` returns nothing), the bootstrap script was interrupted. Rerun it manually:

```
bash /workspaces/agtech_ros2/.devcontainer/bootstrap.sh
```

This is idempotent — it's safe to run it twice.

8. The Scavenger Hunt File Is Missing

If `/root/mystery_folder/` doesn't exist, the bootstrap script failed to create it. Rerun the bootstrap (item 7) or create it manually:

```
mkdir -p /root/mystery_folder/level_1
echo 'CODE: KERNEL_OF_TRUTH' > /root/mystery_folder/level_1/.hidden_payload.txt
```

9. The "Nuclear Option"

If the container is broken beyond repair: press **F1** → **"Dev Containers: Rebuild Container"**. This re-runs the bootstrap from scratch and takes 5 minutes.

Technical Appendix: Security and OS Architecture

Safety Note on `xhost`

To facilitate graphical output, we use `xhost +local:root`. This relaxes X11 permissions. It is safe for local development but resets on reboot.

Why Linux differs from Windows

Windows uses **WSLg** (automated graphics bridge). Native Linux requires manually bridging the X11 Unix socket and defining the `DISPLAY` variable to map the container to the hardware monitor.

Appendix B: Windows Setup Deep Dive (WSL)

Attention Windows users: If you are installing Docker Desktop for the first time, you may encounter issues where the graphics window (Part 5) does not open.

This happens because Docker Desktop provides the *engine*, but it relies on Windows Subsystem for Linux (WSL) to provide the *environment*. Follow these steps to ensure a robust foundation.

Step 1: Install Ubuntu on WSL

1. Open **PowerShell** as an **administrator**.
2. Run the following command to install the standard Ubuntu environment:

```
wsl --install -d Ubuntu-22.04

```

3. **Restart your computer** if prompted.
4. After the restart, a terminal window will open automatically. Create your **UNIX username** and **password** when prompted.

Step 2: Set Ubuntu as Default

We must explicitly tell Windows that Ubuntu is our primary Linux environment.

1. Open **PowerShell** (as a regular user).
2. Set Ubuntu as the default:

```
wsl --set-default Ubuntu-22.04

```

3. **Verify:** Run `wsl --list --verbose`. Ensure there is an asterisk (*) next to Ubuntu-22.04.

Step 3: The "Socket Check" (Critical Concept)

Before we verify the graphics, let's look at *how* Windows talks to Linux. In Linux, "everything is a file" — even the hardware connection to your screen.

1. In your Ubuntu terminal, run this command to look inside the temporary directory:

```
ls -la /tmp/.X11-unix

```

2. **What to look for:** You should see a file named `X0` (capital X, zero).
3. **The lesson:** This `X0` is the "socket" (or plug). If this file exists, your graphics pipeline is physically connected. If it is missing, graphical applications have nowhere to go.

Step 4: The "Eye Test" (Visual Verification)

Now that we know the socket exists, let's use it.

1. Update your system and install the testing apps:

```
sudo apt update && sudo apt install x11-apps -y

```

2. Run the test application:

```
xeyes

```

3. **Success:** A pair of eyes should appear and follow your mouse. Close them and proceed to Part 3.